VÕ TIẾN
Thảo luận kiến thức CNTT trường BK về KHMT(CScience), KTMT(CEngineering)
https://www.facebook.com/groups/khmt.ktmt.cse.bku

Cấu Trúc Dữ Liệu và Giải Thuật (DSA)

DSA - HK243

Cuối Kì

Thảo luận kiến thức CNTT trường BK
về KHMT(CScience), KTMT(CEngineering)
https://www.facebook.com/groups/khmt.ktmt.cse.bku

TP. HỒ CHÍ MINH, THÁNG 7/2025

Mục lục

1 Lý Thuyết Graph **2**
1.1 Các loại biểu diễn đồ thị 2
1.2 DFS và BFS 3
1.3 Topo Sort 3
1.4 Câu khung nhỏ nhất và dijkstra 4

2 Câu hỏi trong các đề thi **6**

1 Lý Thuyết Graph

1.1 Các loại biểu diễn đồ thị

1. Ma trận kề (Adjacency Matrix)

- Sử dụng ma trận vuông kích thước $n \times n$ với n là số đỉnh.
- Phần tử $a[i][j] = 1$ nếu tồn tại cạnh từ i đến j, ngược lại là 0.
- Với đồ thị có trọng số, $a[i][j]$ là trọng số cạnh hoặc vô cực nếu không có cạnh.

Ưu điểm:

- Truy xuất cạnh nhanh: $O(1)$.
- Dễ cài đặt.

Nhược điểm:

- Tốn bộ nhớ: $O(n^2)$.

2. Danh sách kề (Adjacency List)

- Mỗi đỉnh có một danh sách các đỉnh kề.
- Với đồ thị có trọng số: lưu cặp `(đỉnh kề, trọng số)`.

Ưu điểm:

- Tiết kiệm bộ nhớ với đồ thị thưa: $O(n + m)$.
- Duyệt các đỉnh kề nhanh.

Nhược điểm:

- Kiểm tra cạnh tồn tại chậm hơn: $O(\text{bậc của đỉnh})$.

3. Danh sách cạnh (Edge List)

- Lưu danh sách các cạnh dưới dạng các cặp (u, v) hoặc bộ ba (u, v, w) nếu có trọng số.

Ưu điểm:

- Đơn giản, dễ sử dụng cho các thuật toán như Kruskal.

Nhược điểm:

- Không tiện cho việc duyệt các đỉnh kề hoặc kiểm tra cạnh tồn tại.

So sánh tổng quan

Biểu diễn	Bộ nhớ	Duyệt đỉnh kề	Kiểm tra cạnh	Phù hợp với
Ma trận kề	$O(n^2)$	Trung bình	Rất nhanh ($O(1)$)	Đồ thị dày
Danh sách kề	$O(n + m)$	Nhanh	Trung bình	Đồ thị thưa
Danh sách cạnh	$O(m)$	Không tiện	Chậm ($O(m)$)	Kruskal, duyệt cạnh

Chọn biểu diễn tùy bài toán:

- Cần kiểm tra nhanh cạnh → Ma trận kề.
- Cần duyệt hàng xóm thường xuyên → Danh sách kề.
- Duyệt hoặc sắp xếp cạnh → Danh sách cạnh.

1.2 DFS và BFS

Ý tưởng DFS:

- DFS là thuật toán duyệt theo chiều sâu.
- Xuất phát từ một đỉnh, ta đi càng sâu càng tốt theo các đỉnh kề chưa được thăm.
- Khi không còn đường để đi, ta quay lui để tiếp tục duyệt các nhánh còn lại.

```
void DFS(int u) {
    visited[u] = true; // Đánh dấu đỉnh u đã được thăm
    for (int v : adj[u]) { // Duyệt các đỉnh kề với u
        if (!visited[v]) {
            DFS(v); // Gọi đệ quy DFS từ đỉnh v
        }
    }
}
```

Ý tưởng BFS:

- BFS là thuật toán duyệt theo chiều rộng.
- Xuất phát từ một đỉnh, ta lần lượt thăm các đỉnh kề trước (các đỉnh gần), sau đó mở rộng dần ra các đỉnh xa hơn.
- BFS thường sử dụng hàng đợi (queue) để lưu các đỉnh cần xử lý.

```
void BFS(int s) {
    queue<int> q;
    visited[s] = true;
    q.push(s); // Bắt đầu từ đỉnh s

    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (!visited[v]) {
                visited[v] = true;
                q.push(v); // Thêm đỉnh kề vào hàng đợi
            }
        }
    }
}
```

1.3 Topo Sort

Ý tưởng DFS:

- Duyệt DFS từ các đỉnh chưa thăm.
- Khi rời khỏi đỉnh (hết nhánh), thêm vào danh sách kết quả.
- Đảo ngược danh sách kết quả để được thứ tự topo.

```
vector<int> topo;
bool visited[MAX];

void dfs(int u) {
    visited[u] = true;
    for (int v : adj[u]) {
        if (!visited[v])
            dfs(v);
```

```
    }
    topo.push_back(u); // thêm sau khi duyệt xong
}

void topological_sort(int n) {
    for (int i = 1; i <= n; ++i) {
        if (!visited[i])
            dfs(i);
    }
    reverse(topo.begin(), topo.end()); // Đảo ngược để được topo đúng
}
```

Ý tưởng BFS:

- Tính bậc vào (in-degree) của mỗi đỉnh.
- Đưa các đỉnh có in-degree = 0 vào hàng đợi.
- Lặp lại: lấy đỉnh ra, giảm in-degree các đỉnh kề, nếu còn đỉnh có in-degree = 0 thì tiếp tục.

```
vector<int> topo;
int indegree[MAX];

void topological_sort(int n) {
    queue<int> q;
    for (int i = 1; i <= n; ++i)
        if (indegree[i] == 0)
            q.push(i);

    while (!q.empty()) {
        int u = q.front(); q.pop();
        topo.push_back(u);

        for (int v : adj[u]) {
            indegree[v]--;
            if (indegree[v] == 0)
                q.push(v);
        }
    }
}
```

1.4 Câu khung nhỏ nhất và dijkstra

1. **Kruskal:**
   - Sắp xếp tất cả các cạnh theo thứ tự tăng dần trọng số.
   - Duyệt từng cạnh và thêm vào cây nếu **không tạo chu trình** (dùng Union-Find để kiểm tra).
   - Dừng khi có đủ `n - 1` cạnh.
2. **Prim:**
   - Xây dựng cây dần dần từ một đỉnh ban đầu.
   - Mỗi bước chọn cạnh có trọng số nhỏ nhất nối từ cây hiện tại đến đỉnh chưa thuộc cây.
   - Dùng hàng đợi ưu tiên để tối ưu hiệu suất.
3. **Dijkstra – Tìm đường đi ngắn nhất từ một đỉnh**
   - Khởi tạo khoảng cách từ nguồn đến mọi đỉnh là vô cùng (`INF`), riêng đỉnh nguồn là `0`.
   - Mỗi bước chọn đỉnh có khoảng cách nhỏ nhất chưa xử lý.

- Cập nhật khoảng cách của các đỉnh kề nếu tìm được đường đi ngắn hơn.
- Lặp lại cho đến khi xử lý hết các đỉnh.

2 Câu hỏi trong các đề thi

1. Cho G là một đồ thị đơn với 20 đỉnh và 8 thành phần liên thông. Nếu xóa một đỉnh của G thì số thành phần liên thông của G sẽ nằm trong đoạn . . . .

   a) 8 và 20 b) 8 và 19 c) 7 và 19 d) 7 và 20

   Đáp Án: C - **Giải thích:** trường hợp nhỏ nhất khi có 1 liên thông có 1 node, trường hợp lớn nhất 7 liên thông có 1 node và liên thông còn lại có hình sao

2. Cho đồ thị có hướng có các cạnh như sau: (A, B, 6), (A, D, 3), (B, C, 2), (C, E, 5), (D, E, 7). Nếu thực hiện thuật toán DFS bắt đầu từ đỉnh A, với ưu tiên theo thứ tự chữ cái tăng dần, thứ tự các đỉnh được duyệt là gì?

   a) A B C E D b) A D E B C c) A B D C E d) A D B C E

   Đáp Án: A - **Giải thích:** Duyệt DFS từ đỉnh A, các đỉnh kề là B và D. Theo thứ tự chữ cái, D được duyệt sau nên ta đi đến B trước. Từ B đi đến C, rồi đến E. Sau đó quay lại A để duyệt D (nếu chưa duyệt). Trình tự là: A → B → C → E → D.

3. Cho đồ thị có hướng có các cạnh như sau: (A, B, 6), (A, D, 3), (B, C, 2), (C, E, 5), (D, E, 7). Nếu thực hiện thuật toán BFS bắt đầu từ đỉnh A, với ưu tiên theo thứ tự chữ cái tăng dần, thứ tự các đỉnh được duyệt là gì?

   a) A B D C E b) A D B C E c) A B C D E d) A D E B C

   Đáp Án: A - **Giải thích:** Duyệt BFS từ A, ta đưa vào hàng đợi các đỉnh kề theo thứ tự chữ cái: B, D. Lần lượt duyệt B → C, rồi D → E. Trình tự là: A → B → D → C → E.

4. Cho đồ thị có hướng có các cạnh như sau: (A, B), (A, C), (B, D), (C, E), (E, D), (E, F). Thực hiện thuật toán **DFS (duyệt theo thứ tự từ điển tăng)** bắt đầu từ đỉnh A. **Khi thuật toán vừa đến đỉnh C, ngăn xếp DFS chứa những đỉnh nào?**

   a) A, B, C b) A, B, D, C c) A, B, D, E, C d) A, C

   Đáp Án: D - **Giải thích:** DFS từ A theo thứ tự từ điển tăng: các đỉnh kề là B và C, nên đi B trước. Từ B đi đến D. Từ D không còn đỉnh nào chưa duyệt, nên quay lại A và duyệt tiếp C. Như vậy, tại thời điểm vừa đến đỉnh C, ta đã hoàn tất nhánh $B \to D$ và quay lại từ A. Do đó, stack chỉ còn $A \to C$.

5. Cho đồ thị có hướng có các cạnh như sau: (A, B), (A, C), (B, D), (C, E), (E, D), (E, F). Thực hiện thuật toán **BFS (duyệt theo thứ tự từ điển tăng)** bắt đầu từ đỉnh A. **Khi thuật toán vừa đến đỉnh C, hàng đợi (queue) chứa những đỉnh nào?**

   a) B, C b) B, D, C c) D, E, F d) D, C

   Đáp Án: A - **Giải thích:** Bắt đầu từ A, BFS sẽ thêm các đỉnh kề theo thứ tự từ điển là B, C vào hàng đợi. Lúc này A được xử lý xong. Hàng đợi là $[B, C]$. Lúc này thuật toán vừa đến đỉnh C, nên hàng đợi đang chứa các đỉnh B và C trước khi C được xử lý.

6. Cho đồ thị có hướng có ma trận kề như sau:

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix}$$

   Bậc vào và bậc ra của đỉnh 2 (tức đỉnh thứ hai) là gì?

   a) Bậc vào = 1, bậc ra = 2 b) Bậc vào = 2, bậc ra = 2
   c) Bậc vào = 2, bậc ra = 1 d) Bậc vào = 1, bậc ra = 1

   Đáp Án: B - **Giải thích:** Đỉnh 2 có hai cạnh đi vào từ đỉnh 1 và đỉnh 4 (theo cột 2 có hai số 1), và hai cạnh đi ra tới đỉnh 3 và 4 (theo hàng 2 có hai số 1). Do đó bậc vào là 2, bậc ra là 2.

7. Cho đồ thị vô hướng có trọng số được biểu diễn bởi danh sách các cạnh như sau: (A, B, 4), (A, C, 2), (B, D, 5), (C, E, 1), (E, D, 3), (E, F, 7), (D, F, 6), (B, E, 8).Hỏi tổng trọng số nhỏ nhất của cây khung nhỏ nhất (Minimum Spanning Tree) là bao nhiêu?

   a) 13 b) 15 c) 16 d) 17

   Đáp Án: C - **Giải thích:** Áp dụng thuật toán Kruskal:

   - Chọn (C, E, 1)
   - Chọn (A, C, 2)
   - Chọn (E, D, 3)
   - Chọn (A, B, 4)
   - Chọn (D, F, 6)

   Tổng trọng số là: $1 + 2 + 3 + 4 + 6 = 16$

8. Cho một đồ thị có V đỉnh và E cạnh, độ phức tạp thời gian của thuật toán duyệt theo chiều sâu (DFS) và duyệt theo chiều rộng (BFS) lần lượt là:

   a) $O(V^2)$ cho cả DFS và BFS
   b) $O(V + E)$ cho cả DFS và BFS
   c) $O(E \log V)$ cho DFS, $O(V \log V)$ cho BFS
   d) $O(V)$ cho DFS, $O(E)$ cho BFS

   Đáp Án: B

9. Cho đồ thị **vô hướng** có các cạnh như sau: (A, B), (A, C), (B, D), (E, F). Thực hiện thuật toán **DFS (duyệt theo thứ tự từ điển tăng)** bắt đầu từ đỉnh A. **Thứ tự duyệt các đỉnh là:**

   a) A, B, D, C, E, F b) A, C, B, D, E, F c) A, B, C, D, F, E d) A, D, B, C, E, F

   Đáp Án: A - **Giải thích:** Từ đỉnh A có 2 đỉnh kề là B và C, duyệt theo thứ tự từ điển nên đi B trước. Từ B đi tiếp đến D. Sau khi quay lui, mới duyệt tiếp C. Các đỉnh E và F không được duyệt vì nằm ở thành phần liên thông khác và duyệt E trước.

10. Cho đồ thị có hướng không chu trình (DAG) với các cạnh như sau: (A, B), (A, C), (B, D), (C, D), (D, E).Khi áp dụng thuật toán Topological Sort (sử dụng BFS với queue), thứ tự nào sau đây KHÔNG phải là một kết quả hợp lệ?

    a) A → B → C → D → E
    b) A → C → B → D → E
    c) A → B → D → C → E
    d) A → C → D → B → E

    Đáp Án: A - Topo Sort đảm bảo rằng nếu có cạnh (u → v), thì u phải đứng trước v trong kết quả.

11. Cho đồ thị có hướng $G(V, E)$ với $V = \{A, B, C, D, E, F\}$ và ma trận kề E như sau:

$$\begin{bmatrix} 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

    Kết quả xuất đồ thị khi thực hiện thuật toán **DFS (Depth-First Search)** bắt đầu từ đỉnh A là gì?

    a) A B C D E F
    b) A B D C E F
    c) A C D B E F
    d) A E D C E F

    Đáp Án: B - đỉnh b được ưu tiên trước khi duyệt

12. Cho đồ thị có hướng $G(V, E)$ với $V = \{A, B, C, D, E, F\}$ và ma trận kề như sau:

$$\begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Hỏi: Những đỉnh nào được duyệt qua để tìm kiếm đường đi từ A đến F bằng thuật toán **BFS (Breath-First Search)**?

a) A E F
b) A B D F
c) A B C D F
d) A B C D E F

Đáp Án: D – **Giải thích:** Từ A có hai cạnh đi B và C, thêm vào hàng đợi theo thứ tự từ điển: B, C. Từ B đi đến E, từ C đi đến D và E. Từ D đi đến F. Như vậy ta duyệt các đỉnh theo thứ tự: $A \to B \to E \to C \to D \to F$.

13. Cho đồ thị vô hướng có n đỉnh được biểu diễn dưới dạng danh sách kề `adj[1..n]`. Viết hàm DFS đệ quy để duyệt từ đỉnh u và đánh dấu các đỉnh đã thăm trong mảng `visited[1..n]`.

```
void DFS(int u) {
    visited[u] = true;
    for (int v : adj[u]) {
        if (!visited[v]) {
            %%% CODE MISSING %%%
        }
    }
}
```

Lựa chọn nào sau đây đúng để thay thế dòng `%%% CODE MISSING %%%`?

a) DFS(v);
b) visited[v] = true;
c) DFS(u);
d) continue;

Đáp Án: A

14. Cho đồ thị vô hướng có n đỉnh và m cạnh, được lưu dưới dạng danh sách kề `adj[1..n]`. Viết chương trình đếm số thành phần liên thông của đồ thị bằng DFS đệ quy.

```
int cnt = 0;
for (int u = 1; u <= n; ++u) {
    if (!visited[u]) {
        %%% CODE MISSING %%%
        ++cnt;
    }
}
```

Lựa chọn nào sau đây đúng để thay thế dòng `%%% CODE MISSING %%%`?

a) DFS(cnt);
b) visited[u] = true;
c) DFS(u);
d) cnt = 0;

Đáp Án: C

Cho 2 câu sau Xét đoạn mã C++ dưới đây dùng để duyệt đồ thị bằng DFS (không đệ quy). Hãy điền vào các vị trí còn thiếu trong đoạn mã.

```
// Giả sử MAX đã được khai báo
vector<int> adj[MAX];
bool visited[MAX];

```

```cpp
void DFS(int start) {
    stack<int> st;
    st.push(start);

    while (!st.empty()) {
        int u = ____A____;
        st.pop();

        if (!visited[u]) {
            visited[u] = true;
            cout << u << " ";

            for (int i = adj[u].size() - 1; i >= 0; --i) {
                int v = ____B____;
                if (!visited[v]) {
                    st.push(v);
                }
            }
        }
    }
}
```

15. Hãy chọn câu đúng để thay vào dòng ___**A**___ nhằm khởi tạo thuật toán đúng.

    a) `st.push(u);` b) `true;` c) `st.pop();` d) `st.top()`

    Đáp Án: D

16. Hãy chọn câu đúng để thay vào dòng ___**B**___ nhằm khởi tạo thuật toán đúng.

    a) `adj[u][U]` b) `adj[u][i]` c) `adj[u+1][i]` d) `adj[u][i+1]`

    Đáp Án: B

Cho 2 câu sau

Xét đoạn mã C++ dưới đây dùng để duyệt đồ thị bằng BFS. Hãy điền vào các vị trí còn thiếu trong đoạn mã.

```cpp
// Giả sử MAX đã được khai báo
vector<int> adj[MAX];
bool visited[MAX];

void BFS(int start) {
    queue<int> q;
    visited[start] = true;
    q.push(start);

    while (!q.empty()) {
        int u = ____A____;
        q.pop();
        cout << u << " ";

        for (int i = 0; i < adj[u].size(); ++i) {
            int v = ____B____;
            if (!visited[v]) {
                visited[v] = true;
                q.push(v);
            }
        }
    }
}
```

17. Hãy chọn câu đúng để thay vào dòng ___**A**___ nhằm lấy phần tử đầu tiên trong hàng đợi.

    a) `q.top();` b) `q.front();` c) `q.pop();` d) `q.back();`

    Đáp Án: B

18. Hãy chọn câu đúng để thay vào dòng ___**B**___ nhằm truy cập đỉnh kề của đỉnh đang xét.

    a) `adj[u][i]` b) `adj[i][u]` c) `adj[u+i]` d) `adj[u][u]`

    Đáp Án: A

19. Cho đoạn code sau đây thực hiện thuật toán sắp xếp tôpô (topological sort) bằng `std::queue`:

```
vector<int> topoSort(int V, vector<int> adj[]) {
    vector<int> inDegree(V, 0);
    for (int u = 0; u < V; u++) {
        for (int v : adj[u]) {
            inDegree[v]++;
        }
    }

    queue<int> q;
    for (int i = 0; i < V; i++) {
        if (inDegree[i] == 0)
            q.push(i);
    }

    vector<int> topoOrder;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        topoOrder.push_back(u);

        for (int v : adj[u]) {
            inDegree[v]--;
            ______TODO______
        }
    }
    return topoOrder;
}
```

Dòng thiếu **TODO** trong đoạn mã là gì?

a) `if (inDegree[v] == 0) q.push(v);` b) `if (inDegree[v] != 0) q.push(v);`
c) `if (inDegree[v] == 1) q.push(v);` d) `if (inDegree[v] < 0) q.push(v);`

Đáp Án: A

Cho 5 câu sau Cho lớp `Graph` được hiện thực dùng ma trận liền kề như sau:

```
class Graph {
private:
    int nover; // số đỉnh trong đồ thị
    int **wm;  // ma trận trọng số
    int minDistance(int dist[], bool sptSet[]) {
        int min = INT_MAX, min_index;
        for (int v = 0; v < nover; v++)
            if (sptSet[v] == false && dist[v] <= min)
                min = dist[v], min_index = v;
        return min_index;
    }
public:
```

```
    void dijkstra(int src) {
        int dist[nover];
        bool sptSet[nover];
        for (int i = 0; i < nover; i++)
            dist[i] = INT_MAX, sptSet[i] = false;
        dist[src] = 0;
        for (int count = 0; count < /*Code1*/; count++) {
            int u = minDistance(dist, sptSet);
            /*Code2*/ = true;
            for (int v = 0; v < nover; v++)
                if (!sptSet[v] && dist[u] != INT_MAX && /*Code3*/)
                    dist[v] = /*Code4*/;
        }
        // printSolution(dist);
    }
};
```

20. Điền vào `Code1`:

    a) `nover` b) `nover - 1` c) `src` d) `src - 1`

    Đáp Án: B - **Giải thích:** Vòng lặp chạy đúng bằng số lượng đỉnh để đảm bảo tất cả các đỉnh đều được xử lý. Vì vậy phải là `nover - 1`.

21. Điền vào `Code2`:

    a) `sptSet[u]` b) `sptSet[src]` c) `sptSet[count]` d) `sptSet[nover]`

    Đáp Án: A - **Giải thích:** Sau khi chọn đỉnh u có khoảng cách nhỏ nhất, ta đánh dấu nó là đã được xử lý bằng cách gán `sptSet[u] = true`.

22. Điền vào `Code3`:

    a) `wm[u][v] < dist[v]` b) `dist[v] + wm[u][v] < dist[u]`
    c) `wm[u][v] < dist[u]` d) `dist[u] + wm[u][v] < dist[v]`

    Đáp Án: D - **Giải thích:** Điều kiện cập nhật khoảng cách là nếu đường đi thông qua u đến v ngắn hơn đường đi hiện tại đến v.

23. Điền vào `Code4`:

    a) `dist[u]` b) `wm[u][v]`
    c) `dist[u] + wm[u][v]` d) `dist[v] + wm[u][v]`

    Đáp Án: C - **Giải thích:** Nếu đi từ src đến v qua u thì tổng quãng đường là `dist[u] + wm[u][v]`.

24. Với chức năng trả về vị trí của đỉnh có trọng số nhỏ nhất và sau đó loại bỏ đỉnh này khỏi danh sách ở mỗi lần lặp, nên áp dụng cấu trúc dữ liệu nào cho hàm `minDistance` thì thích hợp nhất để giảm độ phức tạp thời gian thực thi?

    a) Đống (*Heap*) b) Chồng (*Stack*)
    c) Danh sách liên kết đơn (*Singly Linked List*) d) AVL

    Đáp Án: A - **Giải thích:** Sử dụng `Heap` (ví dụ: hàng đợi ưu tiên) sẽ giúp lấy đỉnh có khoảng cách nhỏ nhất trong thời gian $\mathcal{O}(\log n)$, giảm độ phức tạp thuật toán từ $\mathcal{O}(V^2)$ xuống $\mathcal{O}((V+E)\log V)$.